**University of Alicante**

Computer Engineering – Specialisation in Computing

VAR: Computer Vision and Robotics

# 3D Gaussian Splatting

for Real-time Rendering and SLAM

*A Comparative Study with Neural Radiance Fields*

**Authors:** Jaime Hernández and Yauheni Yatsevivh

**Date:** March 2026

# Contents

**1 Introduction** **2**
1.1 Novel View Synthesis . . . 2
1.2 Neural Radiance Fields (NeRF) . . . 2
1.3 3D Gaussian Splatting (3DGS) . . . 2
1.4 Comparative Analysis: NeRF vs. 3DGS . . . 3

**2 Mathematical Representation of 3D Gaussians** **4**
2.1 The 3D Gaussian Kernel . . . 4
2.2 Gaussian Primitive . . . 4
2.3 Scene Representation and Parameter Count . . . 4

**3 Covariance Factorisation and Rotation** **5**
3.1 The RS Factorisation . . . 5
3.2 Quaternion Representation of Rotation . . . 5

**4 Colour Modelling via Spherical Harmonics** **6**
4.1 Motivation . . . 6
4.2 View-Dependent Colour . . . 6
4.3 Degree Selection . . . 6

**5 Alpha Blending and Accumulation Rendering** **7**
5.1 Alpha Compositing . . . 7
5.2 Per-Gaussian Opacity at a Pixel . . . 7
5.3 Incremental Accumulation . . . 7
5.4 Connection to NeRF Volume Rendering . . . 7

**6 Optimisation Mechanisms: Densification, Pruning, and Opacity** **8**
6.1 Training Objective . . . 8
6.2 Optimiser . . . 8
6.3 Adaptive Density Control . . . 8
6.4 Pruning . . . 8
6.5 Periodic Opacity Reset . . . 8

**7 3D Gaussian Splatting for SLAM** **9**
7.1 Simultaneous Localisation and Mapping . . . 9
7.2 Integration of 3DGS into the SLAM Pipeline . . . 9
7.3 Notable 3DGS-SLAM Systems . . . 10
7.4 Advantages and Open Challenges . . . 10

# 1 Introduction

## 1.1 Novel View Synthesis

Novel view synthesis (NVS) aims to generate photorealistic images from arbitrary viewpoints given a set of images captured from known camera positions. Applications include virtual reality, autonomous driving, and digital heritage preservation. Deep learning has replaced traditional multi-view stereo methods with learned representations, and two have emerged as state-of-the-art: **NeRF** [3] and **3DGS** [2].

## 1.2 Neural Radiance Fields (NeRF)

NeRF represents a scene as a continuous volumetric function:

$$F_\theta : (\mathbf{x}, \mathbf{d}) \longrightarrow (\mathbf{c}, \sigma) \tag{1}$$

where $F_\theta$ is an MLP mapping position $\mathbf{x} \in \mathbb{R}^3$ and viewing direction $\mathbf{d}$ to colour $\mathbf{c}$ and density $\sigma$. A pixel colour is computed by integrating along the ray:

$$\hat{C}(\mathbf{r}) = \int_{t_n}^{t_f} T(t)\,\sigma(\mathbf{r}(t))\,\mathbf{c}(\mathbf{r}(t), \mathbf{d})\,\mathrm{d}t, \quad T(t) = \exp\left(-\int_{t_n}^{t} \sigma(\mathbf{r}(s))\,\mathrm{d}s\right) \tag{2}$$

**Limitations.** NeRF requires 1–2 days of training and renders at 5–30 s/frame due to the cost of per-pixel MLP evaluations.

## 1.3 3D Gaussian Splatting (3DGS)

3DGS represents the scene as a set of *explicit, learnable 3D Gaussian primitives*. Each Gaussian $\mathcal{G}_i$ stores a position $\boldsymbol{\mu}_i \in \mathbb{R}^3$, a covariance $\boldsymbol{\Sigma}_i$, an opacity $o_i$, and view-dependent colour as spherical harmonic (SH) coefficients. Rendering is performed by projecting each 3D Gaussian onto the image plane (splatting) and compositing the resulting 2D splats via differentiable alpha blending on the GPU, achieving >90 fps at 1080p. Gaussians are initialised from a SfM sparse point cloud and optimised end-to-end with adaptive density control (Section 6).

## 1.4 Comparative Analysis: NeRF vs. 3DGS

Table 1: Qualitative comparison between NeRF and 3DGS.

| **Criterion** | **NeRF** | **3D Gaussian Splatting** |
|---|---|---|
| Representation | Implicit (MLP) | Explicit (Gaussian primitives) |
| Rendering method | Ray casting + volume integration | GPU rasterisation (splatting) |
| Render speed | <1–30 fps | >90 fps (consumer GPU) |
| Training time | 1–2 days; 5 min (Instant-NGP) | 20–40 minutes |
| Memory (inference) | Low (MLP weights) | High (∼1 GB/scene) |
| Editability | Difficult | Easy (explicit geometry) |

NeRF's implicit representation handles smooth geometry naturally; 3DGS trades spatial continuity for hardware-accelerated rendering and explicit editability.

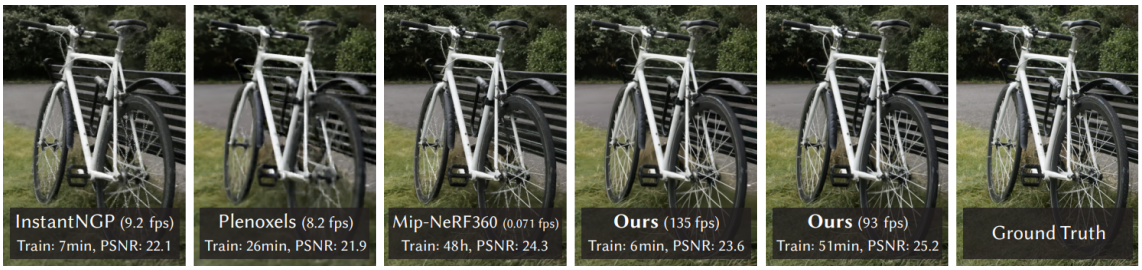


Figure 1: Photo-realistic novel views synthesised with 3D Gaussian Splatting at >90 fps on a consumer GPU. Each scene is reconstructed from a set of posed photographs; the Gaussian map supports real-time rendering from any viewpoint. Image from Kerbl et al. [2].

The remainder of this document covers the mathematical foundations of 3DGS: geometric representation (Sections 2–**??**), the rendering pipeline (Sections **??**–5), the optimisation strategy (Section 6), and the extension to SLAM (Section 7).

# 2 Mathematical Representation of 3D Gaussians

## 2.1 The 3D Gaussian Kernel

A 3D Gaussian is defined by its unnormalised influence kernel:

$$\boxed{\mathcal{G}(\mathbf{x}\,;\,\boldsymbol{\mu},\boldsymbol{\Sigma}) = \exp\left(-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^\top\boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})\right)} \tag{3}$$

where $\boldsymbol{\mu} \in \mathbb{R}^3$ is the centre and $\boldsymbol{\Sigma} \in \mathbb{R}^{3\times 3}$ is a symmetric positive semi-definite (SPSD) covariance matrix. The kernel equals 1 at $\mathbf{x} = \boldsymbol{\mu}$ and decays with the Mahalanobis distance; the locus $\mathcal{G} = e^{-1/2}$ traces the *one-sigma ellipsoid*.

## 2.2 Gaussian Primitive

**Gaussian Primitive**

Each primitive $\mathcal{G}_i$ is an oriented, anisotropic ellipsoid characterised by:

- $\boldsymbol{\mu}_i \in \mathbb{R}^3$ – **position**,
- $\boldsymbol{\Sigma}_i \in \mathbb{R}^{3\times 3}$ – **shape and orientation** (eigenvalues = squared semi-axis lengths; eigenvectors = principal axes),
- $o_i \in [0,1]$ – **opacity**,
- $\mathbf{f}_i \in \mathbb{R}^{(L+1)^2\times 3}$ – **colour** as SH coefficients (Section 4).

## 2.3 Scene Representation and Parameter Count

A scene is a finite mixture of $N$ primitives:

$$\mathcal{S} = \left\{(\boldsymbol{\mu}_i,\, \boldsymbol{\Sigma}_i,\, o_i,\, \mathbf{f}_i)\right\}_{i=1}^{N} \tag{4}$$

Typical scenes contain $N \in [10^5, 10^7]$ Gaussians stored as flat GPU tensors. For SH degree $L = 3$ each Gaussian has 59 learnable parameters $(3+4+3+1+48)$, so $3\times 10^6$ Gaussians require $\approx 700$ MB in `float32`.

# 3 Covariance Factorisation and Rotation

## 3.1 The RS Factorisation

To guarantee SPSD throughout optimisation, 3DGS parametrises $\mathbf{\Sigma}$ as:

$$\boxed{\mathbf{\Sigma} = \mathbf{R}\,\mathbf{S}\,\mathbf{S}^\top\mathbf{R}^\top} \tag{5}$$

where $\mathbf{R} \in SO(3)$ is a rotation matrix and $\mathbf{S} = \mathrm{diag}(s_1, s_2, s_3)$ with $s_k > 0$. SPSD is guaranteed since $\mathbf{v}^\top\mathbf{\Sigma}\,\mathbf{v} = \|\mathbf{S}^\top\mathbf{R}^\top\mathbf{v}\|^2 \geq 0$ for all $\mathbf{v}$. Expanded:

$$\mathbf{\Sigma} = \mathbf{R}\begin{pmatrix} s_1^2 & 0 & 0 \\ 0 & s_2^2 & 0 \\ 0 & 0 & s_3^2 \end{pmatrix}\mathbf{R}^\top \tag{6}$$

The columns of $\mathbf{R}$ are the eigenvectors of $\mathbf{\Sigma}$ and $s_k^2$ are the eigenvalues, so $s_k$ are the principal semi-axis lengths.

## 3.2 Quaternion Representation of Rotation

$\mathbf{R}$ is stored as a unit quaternion $\mathbf{q} = (q_w, q_x, q_y, q_z)$, $\|\mathbf{q}\| = 1$, which uses only 4 parameters and avoids gimbal lock:

$$\mathbf{R}(\mathbf{q}) = \begin{pmatrix} 1 - 2(q_y^2 + q_z^2) & 2(q_x q_y - q_w q_z) & 2(q_x q_z + q_w q_y) \\ 2(q_x q_y + q_w q_z) & 1 - 2(q_x^2 + q_z^2) & 2(q_y q_z - q_w q_x) \\ 2(q_x q_z - q_w q_y) & 2(q_y q_z + q_w q_x) & 1 - 2(q_x^2 + q_y^2) \end{pmatrix} \tag{7}$$

After each gradient step $\mathbf{q}$ is renormalised: $\mathbf{q} \leftarrow \mathbf{q}/\|\mathbf{q}\|$.

# 4 Colour Modelling via Spherical Harmonics

## 4.1 Motivation

Surfaces exhibit view-dependent appearance (specular highlights, metallic reflections). In 3DGS each Gaussian encodes its view-dependent colour compactly using **spherical harmonics (SH)**, an orthonormal basis on the unit sphere $S^2$

## 4.2 View-Dependent Colour

For Gaussian $\mathcal{G}_i$ and channel $k \in \{R, G, B\}$, the colour in direction $\mathbf{d}$ is:

$$\boxed{c_{i,k}(\mathbf{d}) = \sum_{l=0}^{L} \sum_{m=-l}^{l} f_{i,k,l}^{m} \, Y_l^m(\mathbf{d})} \tag{8}$$

where $f_{i,k,l}^{m}$ are learned coefficients. In matrix form: $\mathbf{c}_i(\mathbf{d}) = \mathbf{F}_i \, \mathbf{y}(\mathbf{d})$, with $\mathbf{y}(\mathbf{d}) \in \mathbb{R}^{(L+1)^2}$ the vector of SH values. SH evaluation is purely polynomial, making it very efficient on the GPU.

## 4.3 Degree Selection

Table 2: SH parameter count by degree.

| **Max Degree $L$** | **Coeffs/channel** | **Total (RGB)** | **Effect** |
|---|---|---|---|
| 0 | 1 | 3 | Constant diffuse |
| 1 | 4 | 12 | Linear shading |
| 2 | 9 | 27 | Quadratic specular |
| 3 | 16 | 48 | Sharp specular (used in 3DGS) |

The original 3DGS paper uses $L = 3$. Training starts at $L = 0$ and unlocks one degree every 1000 iterations, stabilising early optimisation by fitting low-frequency geometry first.

# 5 Alpha Blending and Accumulation Rendering

## 5.1 Alpha Compositing

Given $N$ colour contributions $\{c_i\}$ with opacities $\{\alpha_i\}$ sorted front-to-back, the final pixel colour is:

$$\boxed{C = \sum_{i=1}^{N} c_i \, \alpha_i \, T_i}, \qquad T_i = \prod_{j=1}^{i-1}(1-\alpha_j), \quad T_1 = 1 \tag{9}$$

$T_i$ is the transmittance reaching layer $i$; $\alpha_i T_i$ is its effective contribution.

## 5.2 Per-Gaussian Opacity at a Pixel

The opacity of Gaussian $\mathcal{G}_i$ at pixel $\mathbf{u}$ combines the learned opacity $o_i$ and the 2D kernel response:

$$\boxed{\alpha_i(\mathbf{u}) = o_i \cdot \exp\!\big(-\tfrac{1}{2}(\mathbf{u}-\boldsymbol{\mu}_i')^\top (\boldsymbol{\Sigma}_i')^{-1}(\mathbf{u}-\boldsymbol{\mu}_i')\big)} \tag{10}$$

## 5.3 Incremental Accumulation

**Algorithm 1** Alpha Blending per Pixel (front-to-back)

**Require:** Sorted Gaussians $\{i_1, \ldots, i_K\}$ covering pixel $\mathbf{u}$
**Ensure:** Pixel colour $C(\mathbf{u})$

```
C ← 0,   T ← 1
for k = 1, ..., K do
    α ← α_{i_k}(u)
    if α < ε_min then continue
    end if
    C ← C + c_{i_k} · α · T;   T ← T · (1 − α)
    if T < ε_stop then break
    end if
end for
return C + T · C_bg
```

$\epsilon_{\text{min}} = \epsilon_{\text{stop}} = 10^{-4}$. Depth and opacity maps are obtained by the same accumulation replacing $\mathbf{c}_i$ with $z_i$ or 1:

$$D(\mathbf{u}) = \sum_i z_i \, \alpha_i \, T_i, \qquad A(\mathbf{u}) = 1 - T_{\text{final}} \tag{11}$$

## 5.4 Connection to NeRF Volume Rendering

The 3DGS formula is the discrete equivalent of the NeRF volume integral. Identifying $\alpha_i = 1 - e^{-\sigma_i \delta_i}$ and $T_i = \prod_{j<i} e^{-\sigma_j \delta_j}$ recovers Eq. (9) exactly. The two methods differ only in *how* $\alpha_i$ is produced, not in the compositing formula.

# 6 Optimisation Mechanisms: Densification, Pruning, and Opacity

## 6.1 Training Objective

All parameters are optimised end-to-end by minimising:

$$\boxed{\mathcal{L} = (1-\lambda)\,\mathcal{L}_1 + \lambda\,\mathcal{L}_{\text{D-SSIM}}}, \quad \lambda = 0.2 \tag{12}$$

$\mathcal{L}_1$ ensures per-pixel accuracy; $\mathcal{L}_{\text{D-SSIM}}$ preserves structural detail by penalising blurry renderings.

## 6.2 Optimiser

Adam with per-group learning rates (position: $1.6 \times 10^{-4}$ decaying to $1.6 \times 10^{-6}$; scale: $5 \times 10^{-3}$; rotation: $10^{-3}$; opacity: $5 \times 10^{-2}$; SH degree 0: $2.5 \times 10^{-3}$; SH 1–3: $2.5 \times 10^{-4}$).

## 6.3 Adaptive Density Control

Every 100 iterations (from iter. 500 to 15 000), the mean position gradient

$$\bar{\nabla}_i = \tfrac{1}{K} \sum_{k=1}^{K} \|\nabla_{\boldsymbol{\mu}_i'} \mathcal{L}\| \tag{13}$$

drives two operations when $\bar{\nabla}_i > \tau_{\text{grad}} = 2 \times 10^{-4}$:

- **Clone** (small Gaussian, $\max s_k < \tau_{\text{size}}$): create an exact copy; both are optimised independently. The region needs more primitives.
- **Split** (large Gaussian, $\max s_k \geq \tau_{\text{size}}$): replace with two Gaussians at scale $s_k/1.6$, positions sampled from $\mathcal{N}(\boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i)$. The primitive covers too large an area.

## 6.4 Pruning

At the same frequency, Gaussians are removed if: (1) opacity $o_i < \epsilon_\alpha = 0.005$, (2) world-space size exceeds $\tau_w$, or (3) projected 2D radius exceeds $\tau_v$ (removes "floaters").

## 6.5 Periodic Opacity Reset

Every 3000 iterations all opacities are reset to $\alpha_{\min} \approx 0.01$. Gaussians that contribute to the scene recover high opacity quickly; redundant ones do not and are pruned at the next pruning step. This prevents ghost Gaussians from accumulating throughout training.

# 7 3D Gaussian Splatting for SLAM

## 7.1 Simultaneous Localisation and Mapping

Simultaneous Localisation and Mapping (SLAM) addresses the chicken-and-egg problem of robotic perception: a moving agent must estimate its own pose (localisation) while concurrently building a map of the environment (mapping), without prior knowledge of either. Classical SLAM pipelines rely on sparse or semi-dense geometric representations such as point clouds, occupancy grids, and keyframe-based pose graphs. These representations are efficient but lack visual fidelity, making them unsuitable for applications that require photorealistic reconstructions, such as augmented reality or human-robot interaction.

3DGS offers a compelling alternative: the same Gaussian primitives used for novel-view synthesis can serve as the scene map, providing simultaneously a dense, photorealistic, and differentiable representation that can be queried or rendered in real time.

## 7.2 Integration of 3DGS into the SLAM Pipeline

A 3DGS-SLAM system alternates between two tightly coupled phases:

1. **Tracking.** Given the current Gaussian map, the camera pose $T_t \in SE(3)$ is estimated by minimising a photometric loss between the rendered frame (obtained by splatting the existing Gaussians from the candidate pose) and the actual camera observation:

$$T_t^* = \arg\min_{T_t} \sum_p \left\| \hat{I}(p;\, T_t,\, \mathcal{G}) - I_t(p) \right\|_1, \tag{14}$$

   where $\hat{I}(p;\, T_t,\, \mathcal{G})$ is the rendered colour at pixel $p$ given map $\mathcal{G}$ and pose $T_t$, and $I_t$ is the observed frame.

2. **Mapping.** Once the pose is fixed, the Gaussian map is updated by (a) spawning new Gaussians at the depth of pixels with high photometric error, and (b) optimising all Gaussian parameters via back-propagation through the differentiable renderer, exactly as in the offline 3DGS pipeline (Section 6).

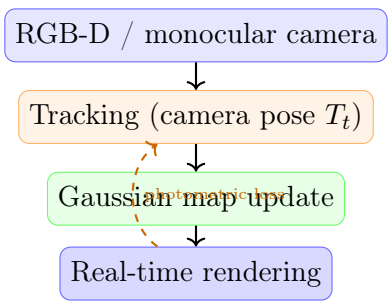


Figure 2: Data-flow in a 3DGS-SLAM system. Tracking and mapping alternate at each keyframe; the differentiable renderer couples both phases.

## 7.3 Notable 3DGS-SLAM Systems

Table 3: Comparison of published 3DGS-SLAM methods.

| System | Input | Key contribution | Speed |
|---|---|---|---|
| SplaTAM [1] | RGB-D | Silhouette-guided densification; first real-time 3DGS SLAM | ∼10 fps mapping |
| Gaussian-SLAM | RGB-D | Online reconstruction with sub-map merging | >10 fps on one GPU |
| MonoGaussian | Monocular RGB | Depth estimated from monocular cues; no depth sensor | Real-time capable |

## 7.4 Advantages and Open Challenges

**Advantages over classical SLAM representations.**

- **Photorealism.** The Gaussian map supports novel-view rendering, enabling downstream tasks such as loop closure via appearance-based place recognition.
- **Differentiability.** Pose optimisation is formulated as gradient descent through the renderer (Eq. 14), without requiring hand-crafted feature descriptors.
- **Real-time rendering.** The GPU-rasterised pipeline delivers >90 fps renders even during online mapping.

**Open challenges.**

- **Memory growth.** The number of Gaussians grows with scene size, making large-scale outdoor SLAM memory-intensive.
- **Loop closure.** Correcting accumulated drift requires updating potentially millions of Gaussians consistently.
- **Dynamic objects.** Moving people or vehicles violate the static scene assumption and produce rendering artefacts.
- **Monocular scale ambiguity.** Without depth, metric scale must be recovered from additional cues or a known object.

**Summary**

3DGS-SLAM unifies photorealistic scene reconstruction and real-time camera tracking in a single differentiable framework, combining the speed of GPU rasterisation with the accuracy of gradient-based pose optimisation. Current systems achieve real-time performance on RGB-D input; extending them to monocular and large-scale outdoor settings remains an active research area.

# References

[1] Nikhil Keetha, Jay Karhade, Krishna Murthy Jatavallabhula, Gengshan Yang, Sebastian Scherer, Deva Ramanan, and Jonathon Luiten. SplaTAM: Splat, track & map 3D Gaussians for dense RGB-D SLAM. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

[2] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3D Gaussian Splatting for real-time radiance field rendering. In *ACM SIGGRAPH 2023 Conference Proceedings*. ACM, 2023.

[3] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. NeRF: Representing scenes as neural radiance fields for view synthesis. In *European Conference on Computer Vision (ECCV)*, pages 405–421. Springer, 2020.